TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



CÔNG NGHỆ PHẦN MỀM

Assignment 2: Từ yêu cầu đến triển khai

BrewLite – Requirements to Deployment

GVHD: Từ Lãng Phiêu

SV: Nguyễn Thành Long – MSSV: .....

Trần Quốc Tài – MSSV: .....

TP. HỒ CHÍ MINH, THÁNG 11/2026

Mục lục

1 Giới thiệu **2**

1.1 Mục tiêu 2

1.2 Khác biệt so với Assignment 1 2

1.3 Nhóm, kho mã và sản phẩm 2

2 Task 1: Khai thác và đặc tả yêu cầu **3**

2.1 Bối cảnh, stakeholder và phạm vi 3

2.2 Yêu cầu chức năng và phi chức năng 3

2.3 Đặc tả use-case trọng tâm: UC-05 Đặt hàng và thanh toán 6

3 Task 2: Mô hình hóa hệ thống **7**

3.1 Activity diagram (Activity-01) 7

3.2 Sequence diagram (Sequence-01) 8

3.3 Class diagram (Class-01) 9

4 Task 3: Thiết kế kiến trúc **10**

4.1 Phương pháp và quyết định 10

4.2 Deployment diagram 11

5 Task 4: Sprint 1 **12**

5.1 Repository và quản lý tài liệu 12

5.2 MVP Sprint 1 và CI 12

6 Task 5: Sprint 2, CI/CD và bàn giao **12**

6.1 MVP hoàn chỉnh 12

6.2 Kiểm thử và kết quả 12

6.3 Triển khai và bàn giao 13

7 Ma trận truy vết **14**

8 Lịch sử Git và thứ tự tạo phẩm **14**

8.1 Đóng góp 15

9 Hạn chế và hướng phát triển **15**

1 Giới thiệu

1.1 Mục tiêu

Assignment 2 tiếp tục đề tài BrewLite của Assignment 1 nhưng chấm *quy trình kỹ thuật phần mềm đầy đủ*: khai thác và đặc tả yêu cầu, mô hình hóa, thiết kế kiến trúc, hiện thực hai Sprint, thiết lập CI/CD và triển khai lên môi trường truy cập được. Đề gồm 5 task, mỗi task 2 điểm, **làm tuần tự**: tạo phẩm của task trước phải có trong lịch sử git trước task sau. Luồng nghiệp vụ tối thiểu giữ nguyên: *xem menu* → *chọn tùy chọn* → *giỏ hàng* → *đặt đơn* → *thanh toán* → *theo dõi đơn*.

1.2 Khác biệt so với Assignment 1

	Assignment 1	**Assignment 2**
Trọng tâm	Hiện thực nghiệp vụ, chia việc theo Scrum	Quy trình từ yêu cầu đến triển khai
Kho mã	Kho riêng, chứng minh phân công từng thành viên	Kho riêng, đẩy lên theo đúng thứ tự tài liệu → mã
Tài liệu	User story, backlog	SRS (FR/NFR, MoSCoW), use-case, 6 sơ đồ PlantUML, ADR, truy vết
Triển khai	Docker Compose chạy cục bộ	CI/CD, production trên Internet, kiểm thử E2E

1.3 Nhóm, kho mã và sản phẩm

- Nhóm: Nguyễn Thành Long (`Longcodepro`) – backend, DevOps, Scrum Master; Trần Quốc Tài (`TaiTran263`) – frontend, tài liệu yêu cầu và truy vết.
- Kho mã: `https://github.com/Longcodepro/Project_SE_Assignment02_HK1_2026_2027` (monorepo `apps/web`, `apps/api`, `docs/`, `tests/`).
- Frontend (Vercel): `https://se-as2-brewlite.vercel.app`. Backend (Render): `https://brewlite-api.onrender.com`, kiểm tra tại `/health`, tài liệu API tại `/docs`. Ngày 14/11/2026 frontend trả 200; backend trả 200 sau khoảng 8 giây (khởi động nguội của gói miễn phí).
- Công nghệ: Next.js 16, NestJS 12, PostgreSQL 16 (Neon) với Prisma, Docker, GitHub Actions.

2 Task 1: Khai thác và đặc tả yêu cầu

2.1 Bối cảnh, stakeholder và phạm vi

Quán đồ uống nhỏ cần nhận đơn và thanh toán không tiền mặt trên điện thoại để giảm xếp hàng. Nguồn sự thật của phần này là `docs/requirements/context-and-scope.md`.

Stakeholder	Mục tiêu chính
Khách hàng	Đặt đồ uống nhanh, đúng ý, không phải chờ ở quầy
Nhân viên quầy, pha chế	Chuẩn bị đúng đơn, giảm sai sót giờ cao điểm
Quản lý cửa hàng	Tăng hiệu suất bán, kiểm soát doanh thu, chống thất thoát
Đơn vị thanh toán (mô phỏng)	Giao dịch toàn vẹn, bảo mật, chính xác
Nhóm phát triển	Sản phẩm đúng kiến trúc, có kiểm thử tự động, bàn giao đúng hạn
Giảng viên nghiệm thu	Đánh giá kỹ thuật, quy trình và tính chạy được

Trong phạm vi MVP (Sprint 1–2): menu theo danh mục, tìm kiếm, ba trạng thái giao diện; chọn size và topping; giỏ hàng Zustand lưu `localStorage`; đăng ký/đăng nhập bằng số điện thoại 10 số, JWT; tạo đơn với giá do server tính; thanh toán mô phỏng Thẻ/Ví; vòng đời đơn, khóa lạc quan, `Idempotency-Key`; tích điểm; lịch sử đơn có polling 5 giây; Docker, CI, CD, production, kiểm thử tự động. **Ngoài phạm vi (Won't):** hoàn tiền về ví nội bộ, SMS/OTP trả phí, quản lý nhân sự và chấm công, đặt bàn.

2.2 Yêu cầu chức năng và phi chức năng

Tài liệu có 21 yêu cầu chức năng (`FR-01`–`FR-21`) và 5 yêu cầu phi chức năng (`NFR-01`–`NFR-05`), xếp mức MoSCoW: 9 Must, 1 Should, 11 Could. Cột "Trạng thái" phản ánh `main` ngày 14/11/2026.

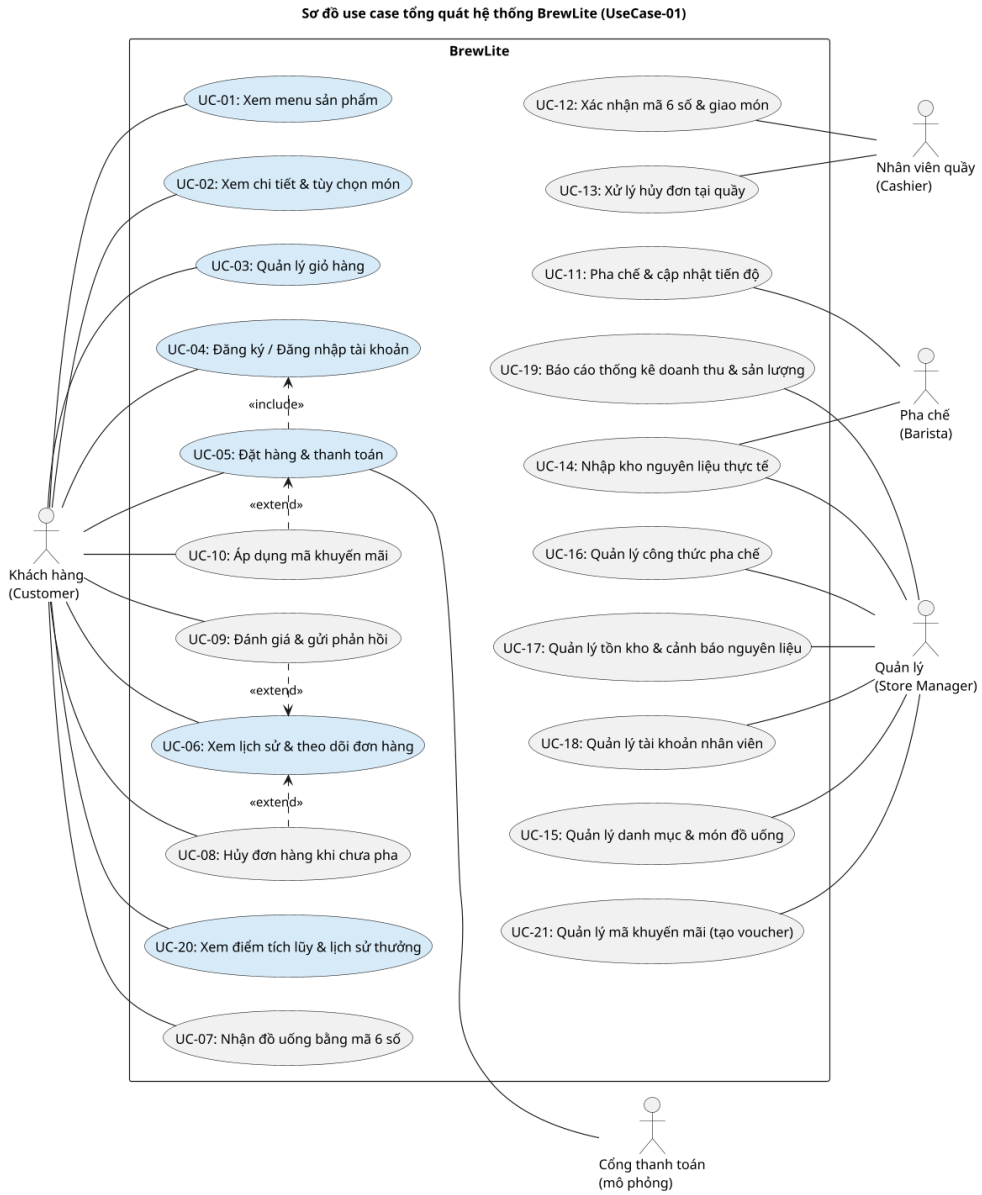
Mã	Yêu cầu	Sprint	MoSCoW	Trạng thái
FR-01	Xem menu theo danh mục, tìm kiếm/lọc, nhãn hết hàng	1	Must	Đã làm
FR-02	Chọn size, topping, xem giá tạm tính	2	Must	Đã làm
FR-03	Giỏ hàng phía client, lưu `localStorage`	2	Must	Đã làm
FR-04	Đăng ký, đăng nhập bằng SĐT, JWT, vai trò	2	Must	Đã làm
FR-05	Tạo đơn, server tự tính tổng tiền	2	Must	Đã làm
FR-06	Thanh toán mô phỏng Thẻ/Ví, không lưu số thẻ	2	Must	Đã làm
FR-07	Lịch sử đơn, theo dõi trạng thái (polling)	2	Must	Đã làm
FR-08	Vòng đời đơn theo bảng chuyển, khóa lạc quan `version`	2	Must	Đã làm
FR-09	Chống gửi trùng bằng `Idempotency-Key`	2	Must	Đã làm
FR-10	Tích điểm 1 điểm/1.000đ, xem số dư và lịch sử	2	Should	Đã làm
FR-11	Voucher (khách áp mã, Manager quản lý mã)	3, 4	Could	Kế hoạch
FR-12	Hủy đơn trước khi pha chế, hoàn tiền/kho/điểm	3	Could	Kế hoạch
FR-13	Đánh giá 1–5 sao cho đơn đã hoàn tất	3	Could	Kế hoạch
FR-14	Báo cáo doanh thu, tỷ lệ hủy, Top 5 món	4	Could	Kế hoạch
FR-15	Kho nguyên liệu, trừ/hoàn kho tự động	4	Could	Kế hoạch
FR-16	Công thức pha chế theo món, size, topping	4	Could	Kế hoạch
FR-17	Cảnh báo nguyên liệu chạm ngưỡng	4	Could	Kế hoạch
FR-18	Quản lý danh mục, món, topping, phụ thu size	4	Could	Kế hoạch
FR-19	Quản lý tài khoản nhân viên	4	Could	Kế hoạch
FR-20	Màn hình KDS cho Barista (`PAID` → `PREPARING` → `READY`)	3	Could	Đã làm
FR-21	Mã nhận hàng 6 số, Cashier hoàn tất đơn	3	Could	Kế hoạch

Bảng 1: Yêu cầu chức năng (nguồn: `context-and-scope.md`, mục 6.3)

Mã	Nhóm	Nội dung
NFR-01	Bảo mật	Không tin giá từ client; `bcryptjs`; JWT 24 giờ, không có giá trị mặc định cho `JWT_SECRET`; không lưu số thẻ thô; không commit secret
NFR-02	Tin cậy, sẵn sàng	Production qua HTTPS, có `GET /health`; kết quả đúng khi gửi trùng hoặc đồng thời
NFR-03	Chất lượng, bảo trì	Mọi PR qua CI; Swagger tại `/docs`; migration có phiên bản
NFR-04	Khả dụng	Mobile-first; menu có đủ ba trạng thái Loading, Empty, Error + Retry
NFR-05	Hiệu năng giao diện	FCP $\leq$ 1,5 s, TBT $\leq$ 200 ms; đo sau khi API đã khởi động

Bảng 2: Yêu cầu phi chức năng

Mỗi yêu cầu có tiêu chí chấp nhận kiểm chứng được ở `requirements-api.md` (phía API) và `requirements-web.md` (phía giao diện); 14 user story `US-01`–`US-14` ở `user-stories.md`.



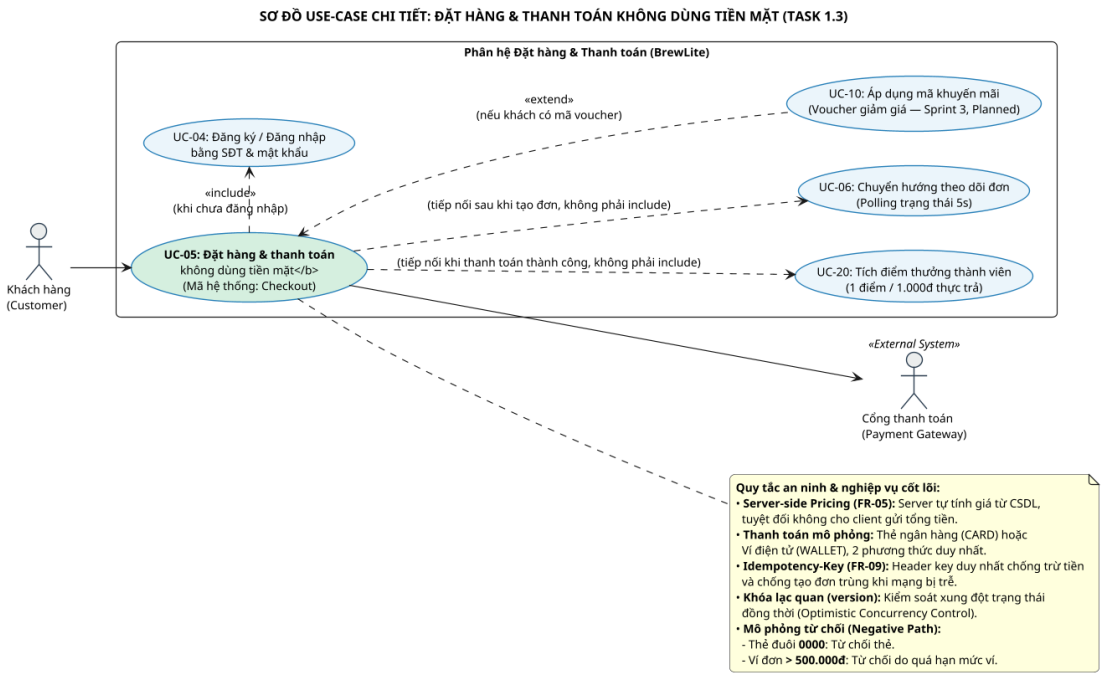
Hình 1: Sơ đồ use-case tổng quát (`docs/diagrams/usecase-overview.puml`)

2.3 Đặc tả use-case trọng tâm: UC-05 Đặt hàng và thanh toán

Mục	Nội dung
Mã, ưu tiên	UC-05, Must; liên kết FR-05, FR-06, FR-08, FR-09, FR-10
Actor	Chính: khách hàng. Phụ: cổng thanh toán mô phỏng; nhân viên pha chế/quầy (nhận đơn sau khi thanh toán)
Trigger	Khách bấm "Tiến hành Đặt hàng" ở `/cart` hoặc "Xác nhận Thanh toán" ở `/checkout`
Tiền điều kiện	Giỏ hàng có ít nhất một món hợp lệ; backend và cơ sở dữ liệu đang chạy (`GET /health` trả 200)
Hậu điều kiện (thành công)	Đơn `PAID` (`version` $=$ 1), có `OrderItem` và `Payment SUCCESS`; cộng điểm 1/1.000đ; xóa giỏ; chuyển sang `/order-success` và polling 5 giây
Hậu điều kiện (thất bại)	Đơn `PAYMENT_FAILED` (API vẫn trả 201 kèm `status`); không trừ tiền, không cộng điểm; giữ nguyên giỏ; được thanh toán lại

Luồng chính. (1) Khách xem giỏ và bấm đặt hàng; hệ thống kiểm tra đăng nhập. (2) Chuyển sang `/checkout`. (3–4) Khách chọn Thẻ (`CARD`) hoặc Ví (`WALLET`). (5) Khách xác nhận; giao diện sinh `Idempotency-Key`, khóa nút. (6) `POST /orders`: xác thực JWT, tính giá ở server, tạo đơn `PENDING`. (7) `POST /payments`. (8) Cổng mô phỏng duyệt thanh toán. (9) Một transaction: đơn $\rightarrow$ `PAID`, ghi `Payment`, cộng điểm. (10) Hiện màn xác nhận, xóa giỏ.

Luồng thay thế và ngoại lệ. Chưa đăng nhập: hiện modal đăng nhập, giữ giỏ rồi tiếp tục (`<<include>>` UC-04). Áp mã giảm giá (`<<extend>>` UC-10, thuộc kế hoạch Sprint 3). Đổi phương thức thanh toán. Thẻ có 4 số cuối `0000` hoặc Ví với đơn trên 500.000đ: `PAYMENT_FAILED`. Đơn đã `PAID` thanh toán tiếp: 409. Thiếu hoặc hết hạn token: 401.



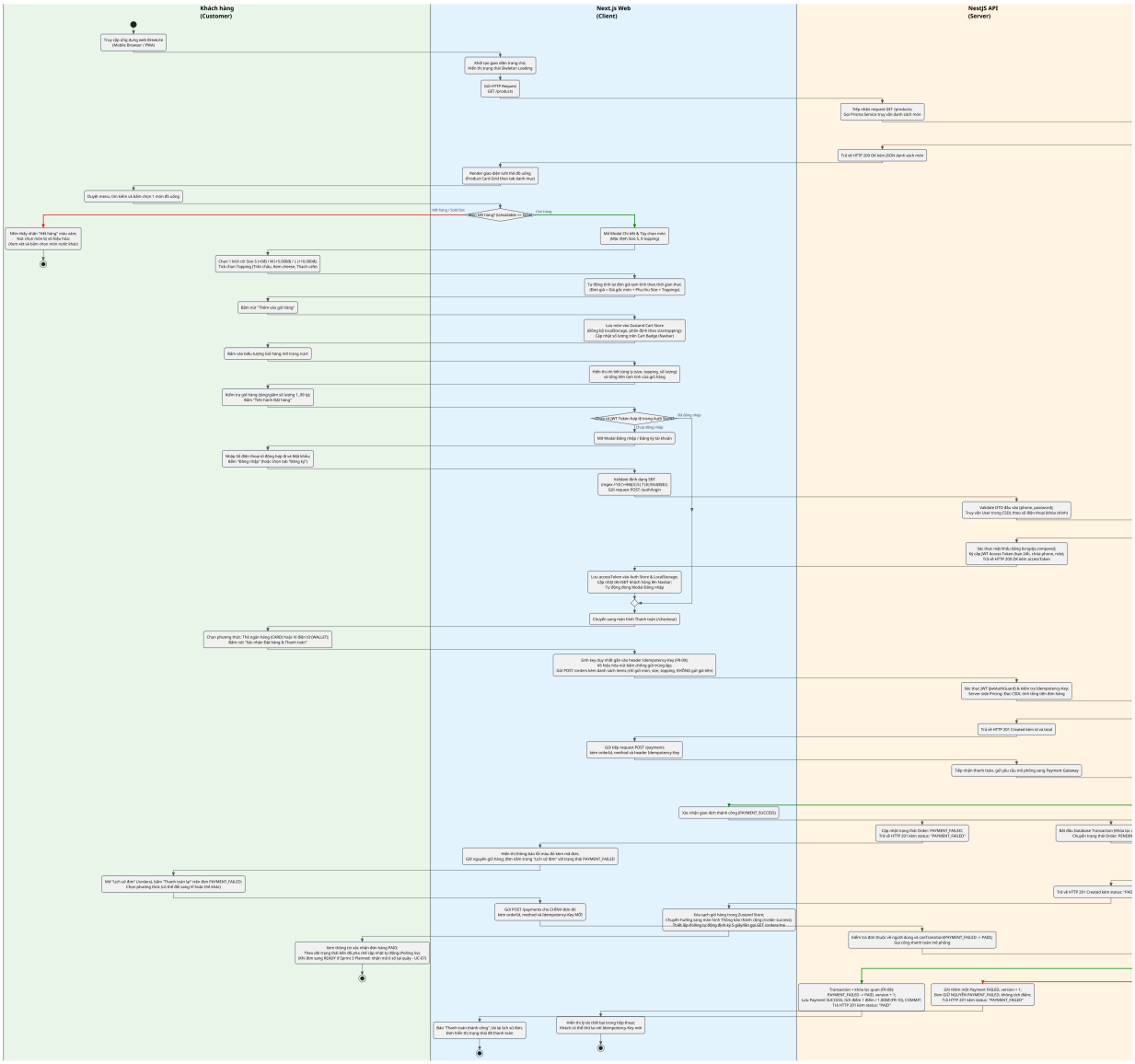
Hình 2: Use-case Đặt hàng và thanh toán (`docs/diagrams/usecase-checkout.puml`)

3 Task 2: Mô hình hóa hệ thống

Cả ba sơ đồ viết bằng PlantUML; tệp `.puml` là nguồn sự thật, `.svg` xuất ra cùng commit. Hình trong báo cáo xuất từ chính các tệp `.puml` đó.

3.1 Activity diagram (Activity-01)

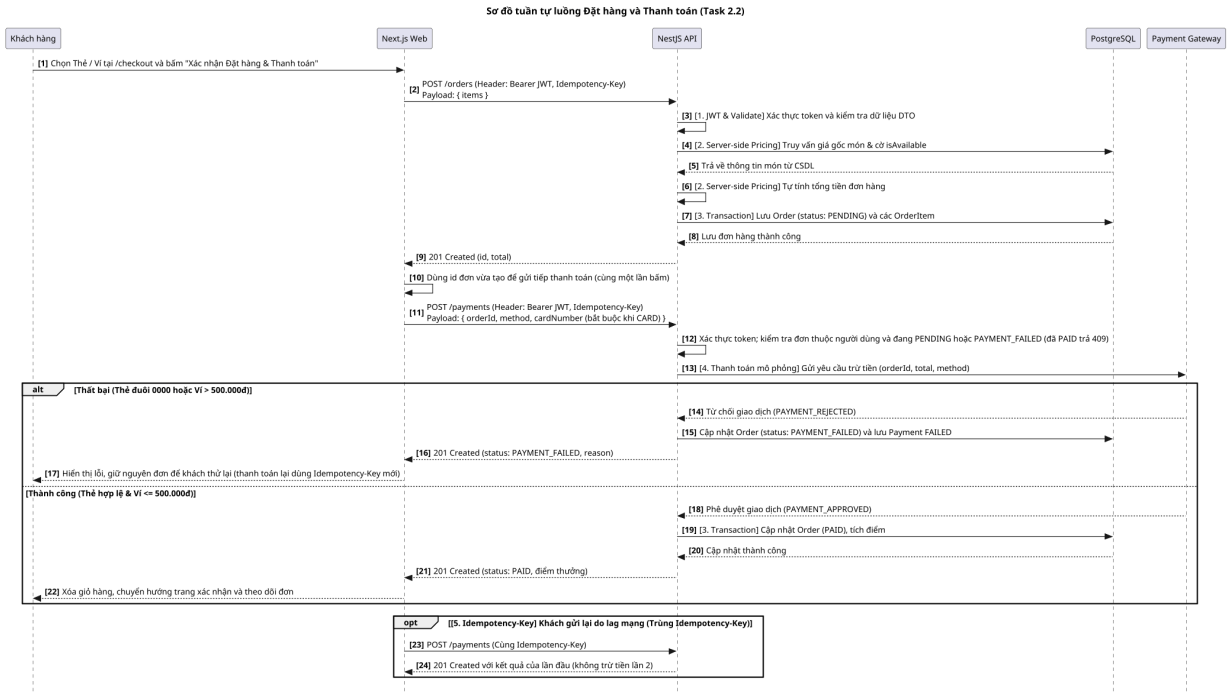
Mô tả luồng từ mở ứng dụng đến theo dõi đơn, gồm các giai đoạn: xem menu, chọn món và size, giỏ hàng, xác thực, thanh toán, theo dõi đơn hàng; có nhánh thanh toán lại đơn `PAYMENT_FAILED`.



Hình 3: Activity-01: luồng đặt món và thanh toán (`docs/diagrams/activity-01-menu-cart-auth.puml`)

3.2 Sequence diagram (Sequence-01)

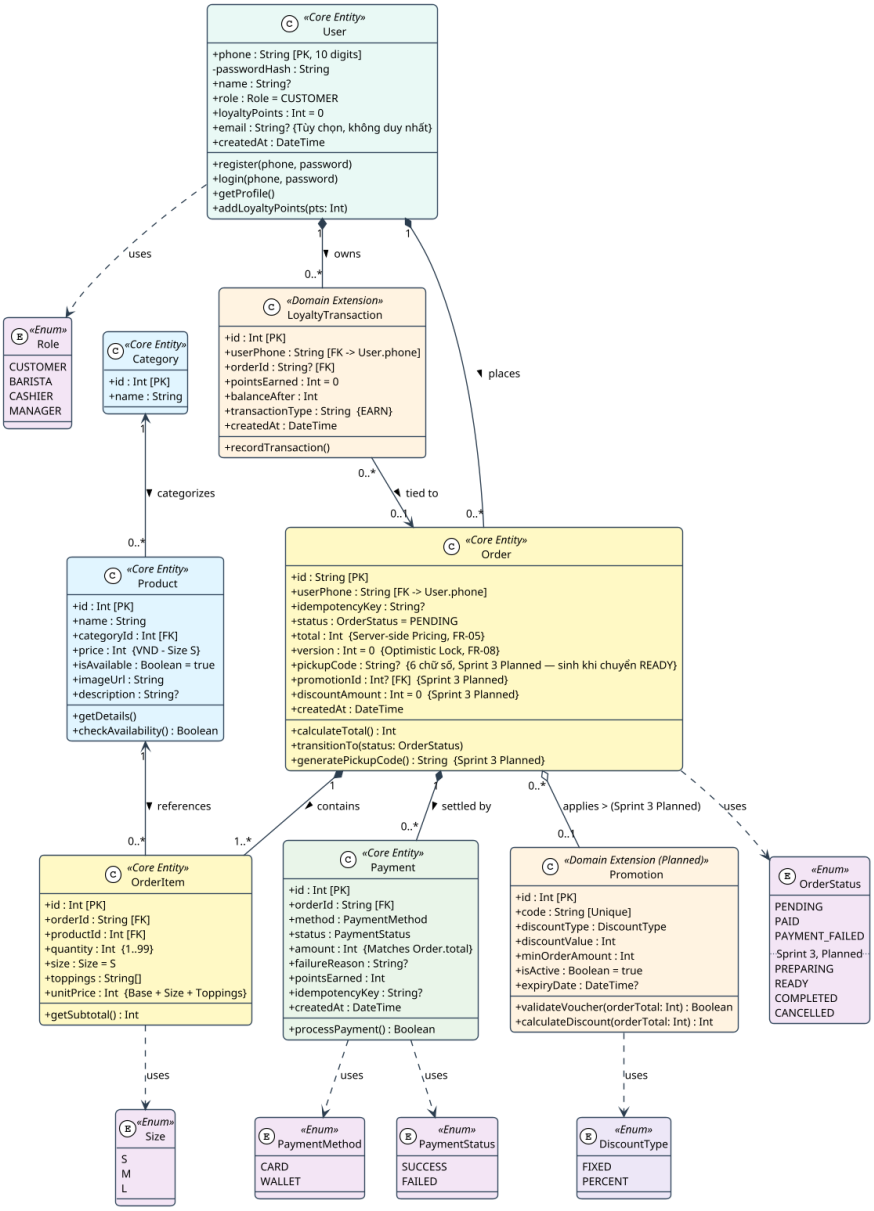
Các thành phần phối hợp: khách hàng, Next.js Web, NestJS API, PostgreSQL và cổng thanh toán mô phỏng. Sơ đồ có nhánh `opt` cho `Idempotency-Key`, nhánh `alt` cho thanh toán thành công/thất bại và đoạn transaction kiểm tra `version`.



Hình 4: Sequence-01: luồng đặt hàng và thanh toán (`docs/diagrams/sequence-checkout.puml`)

3.3 Class diagram (Class-01)

Mô hình miền gồm các lớp lõi `User`, `Category`, `Product`, `Order`, `OrderItem`, `Payment`, các enum `Role`, `OrderStatus`, `Size`, `PaymentMethod`, `PaymentStatus`, `DiscountType`, và hai lớp mở rộng `Promotion` (kế hoạch), `LoyaltyTransaction` (mức miền, chưa là bảng ở MVP). Tên lớp, enum và cột bám theo `apps/api/prisma/schema.prisma`.



Hình 5: Class-01: mô hình miền (`docs/diagrams/class-domain.puml`)

4 Task 3: Thiết kế kiến trúc

4.1 Phương pháp và quyết định

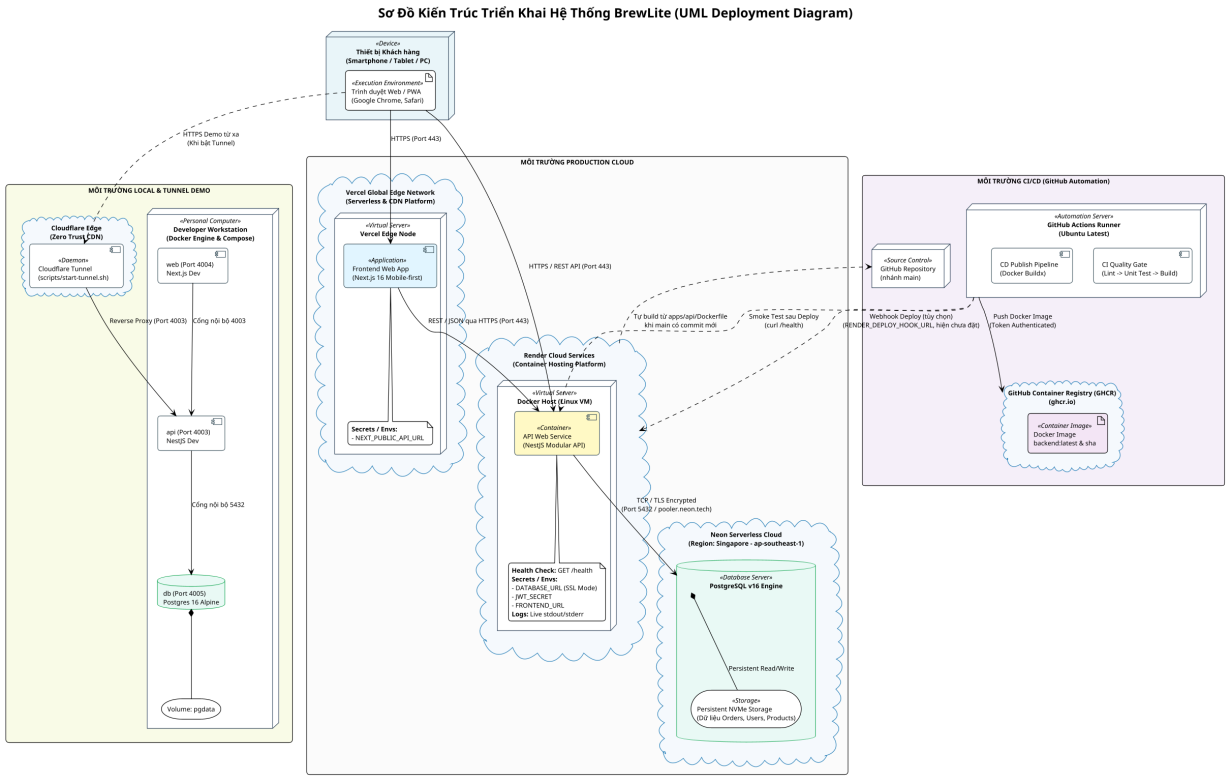
Backend theo mô hình *Modular Monolith* phân lớp Controller → Service → Prisma, các module: `auth`, `products`, `orders`, `payments`, `loyalty`, `kds`, `prisma`. Bốn nguyên tắc: module hóa theo miền nghiệp vụ; server nắm quyền định giá; nguyên tử hóa bằng transaction; chống gửi trùng. **Frontend** dùng Next.js App Router, mobile-first, Zustand (`useCartStore`, `useAuthStore`), Axios có interceptor gắn Bearer token, cho phép đổi URL backend ngay trên giao diện.

ADR	Quyết định	Lý do và đánh đổi
001	Monorepo (`apps/web`, `apps/api`, `docs`, `tests`)	Một PR cho tính năng xuyên tầng, tài liệu cạnh mã, một pipeline; đổi lại lịch sử chung xen kẽ và phải tự đồng bộ hai `package-lock.json`
002	Modular Monolith + phân lớp	Giao dịch ACID cho tạo đơn, thanh toán, cộng điểm; chi phí thấp, dễ tách sau; đổi lại dùng chung một tiến trình, cần kỷ luật tránh import chéo module
003	PostgreSQL + Prisma	Dữ liệu quan hệ chặt, schema tập trung, migration có phiên bản, type-safe; đổi lại truy vấn thống kê phức tạp có thể cần SQL thô và có độ trễ khởi động
004	Zustand + `persist`	Nhẹ, ít re-render, tự đồng bộ `localStorage`; đổi lại DevTools yếu hơn Redux, cần xử lý hydration trên Next.js
005	`Idempotency-Key`	Chặn tạo đơn và trừ tiền hai lần khi mạng chập chờn; đổi lại frontend phải luôn gửi header, tốn thêm chỉ mục duy nhất

Bảng 3: Các quyết định kiến trúc (`docs/architecture/adr/`)

4.2 Deployment diagram

Sơ đồ có ba ranh giới: máy phát triển (Docker Compose), CI trên GitHub Actions và production (Vercel, Render, Neon).



Hình 6: Deployment diagram (`docs/diagrams/deployment.puml`)

5 Task 4: Sprint 1

5.1 Repository và quản lý tài liệu

Monorepo theo cấu trúc bắt buộc của đề: `apps/web`, `apps/api`, `docs/requirements`, `docs/diagrams`, `docs/architecture`, `tests/`, `.github/workflows/`. Mỗi tạo phẩm một nhánh, một Pull Request, một issue (tên nhánh `tai/task-1.1-...`, `long/task-1-2b-...`); commit dạng `docs(task-x.y): ...`, `feat(task-x.y): ...`. Mỗi sơ đồ `.puml` có chú thích đầu tệp ghi tên, mã (`Activity-01`, `Sequence-01`, `Class-01`) và mã FR/UC liên quan.

5.2 MVP Sprint 1 và CI

Sprint 1 (29/10 – 05/11/2026) bàn giao `GET /products` và giao diện Menu mobile-first có ba trạng thái (Skeleton Loading, Empty, Error + Retry). Logic size/topping chuyển sang Sprint 2 để tích hợp với giỏ hàng và định giá phía server. Pipeline CI (`.github/workflows/ci.yml`) chạy ở mọi Pull Request, gồm bốn job: (1) backend: lint `oxlint`, build NestJS, Vitest; (2) frontend: lint `eslint`, Vitest, build Next.js; (3) triển khai và smoke test; (4) E2E luồng đặt hàng bằng Playwright. Job nào lỗi thì chặn merge.

6 Task 5: Sprint 2, CI/CD và bàn giao

6.1 MVP hoàn chỉnh

Sprint 2 (06/11 – 10/11/2026) hoàn thiện luồng giỏ hàng $\rightarrow$ thanh toán $\rightarrow$ tích điểm: modal size/topping tính giá thời gian thực, giỏ hàng bền vững, đăng ký/đăng nhập số điện thoại, màn thanh toán Thẻ/Ví có chống gửi trùng, lịch sử đơn polling 5 giây; phía API gồm `POST /orders` (định giá server), `POST /payments`, `GET /orders/me`, `GET /loyalty/me`, `GET /health`. Ngoài MVP, `main` còn có phần nền Sprint 3 đã merge: bốn trạng thái đơn mới (`PREPARING`, `READY`, `COMPLETED`, `CANCELLED`), bốn vai trò (`CUSTOMER`, `BARISTA`, `CASHIER`, `MANAGER`) và màn hình KDS cho Barista (FR-20).

6.2 Kiểm thử và kết quả

Số liệu chạy trên bản `origin/main` ngày 14/11/2026 (commit `0e4d14e`).

Hạng mục	Nội dung	Kết quả
Backend	`oxlint`, `nest build`	không lỗi
Unit test API	Vitest, 11 file	112/112 qua
Unit test web	Vitest (giỏ hàng, bảng giá so khớp server, modal), 5 file	32/32 qua
E2E	Playwright, `tests/e2e/checkout.spec.ts`: thẻ thành công, thẻ `0000` bị từ chối, 401 khi thiếu token, Ví $\leq$ 500.000đ, Ví $>$ 500.000đ, thanh toán lại đơn `PAYMENT_FAILED`, bấm xác nhận liên tiếp chỉ tạo một đơn, gửi lại cùng `Idempotency-Key`	8 kịch bản
CI trên `main`	bốn job như mục 5.2	xanh
CD trên `main`	đóng gói ảnh Docker `apps/api` lên GHCR; kích hoạt Render và smoke test `/health` nếu đã đặt secret	xanh

Lint, build và Vitest của hai ứng dụng được chạy lại trên bản sao sạch của `origin/main`; riêng 8 kịch bản E2E được đếm từ mã nguồn, kết quả chạy lấy từ job E2E của CI (trạng thái chung của CI trên `main` là xanh).

6.3 Triển khai và bàn giao

CI/CD. CD chạy khi merge vào `main`: build ảnh từ `apps/api/Dockerfile`, gắn thẻ `latest` và mã commit ngắn, đẩy lên GHCR. Render kết nối GitHub và tự build lại khi `main` đổi; lệnh khởi động chạy `prisma migrate deploy`, `prisma db seed` rồi `node dist/main`. **Môi trường.** Frontend trên Vercel, backend trên Render, cơ sở dữ liệu PostgreSQL 16 trên Neon (Singapore, bắt buộc `sslmode=require`); Cloudflare Tunnel làm đường demo dự phòng. Biến môi trường chỉ ghi tên trong tài liệu, không có giá trị bí mật.

Đo hiệu năng (Lighthouse 12.8.2, di động, ngày 13/11/2026 trên production).

Trang	Perf.	A11y	Best Pr.	FCP (giữa)	TBT lớn nhất
`/`	97	92	100	1,2 s	80 ms
`/cart`	97	95	100	1,2 s	70 ms
`/checkout`	97	95	100	1,2 s	70 ms

Bảng 4: Giá trị giữa của 3 lượt đo; CLS = 0 ở cả ba trang

FCP đạt mục tiêu 1,5 s ở giá trị giữa, nhưng 3 trong 9 lượt đo vượt (2,1 s, 1,8 s, 1,8 s), và điều kiện giả lập của Lighthouse chậm hơn 4G thực. Accessibility chưa đạt 100: còn lỗi tương phản màu ở `/` và liên kết không có tên mô tả ở `/`, `/cart`. Vì vậy NFR-05 mới được đo một phần.

7 Ma trận truy vết

Ma trận đầy đủ cho 21 FR và 5 NFR nằm ở `docs/traceability-matrix.md`. Bảng dưới trích các yêu cầu đã hiện thực (MVP), theo mẫu của đề: yêu cầu → use-case/user story → sơ đồ → mã → kiểm thử.

FR	UC / US	Sơ đồ	Mã	Kiểm thử
01	UC-01, US-01	Activity-01, Class-01	`GET /products`; `page.tsx`, `ProductCard.tsx`	`products.controller.spec.ts`, `swagger.spec.ts`, Postman
02	UC-02, US-02	Activity-01, Class-01	`ProductDetailModal.tsx`; `orders/pricing.ts`	`orders.controller.spec.ts`; unit test web
03	UC-03, US-03	Activity-01, ADR-004	`cart/page.tsx`, `useCartStore.ts`	E2E, unit test `useCartStore`
04	UC-04, US-04	Activity-01, Sequence-01	`POST /auth/register`, `/auth/login`; `jwt-auth.guard.ts`	`auth.controller.spec.ts`
05	UC-05, US-05	Sequence-01, Class-01	`POST /orders`; `OrdersService.create`	`orders.controller.spec.ts`, E2E
06	UC-05, US-06	Sequence-01 (`alt`)	`POST /payments`; `mock-gateway.ts`	`payments.controller.spec.ts`, E2E
07	UC-06, US-07	Activity-01	`GET /orders/me`; `orders/page.tsx`	`orders.controller.spec.ts`, E2E
08	UC-05, US-08	Sequence-01	`order-status.ts`; `PaymentsService.pay`	`order-status.spec.ts`, `payments.controller.spec.ts`
09	UC-05, US-09	Sequence-01 (`opt`), ADR-005	`common/idempotency.ts`	`orders/payments.controller.spec.ts`, E2E
10	UC-20, US-10	Sequence-01, Class-01	`payments/loyalty.ts`; `GET /loyalty/me`	`loyalty.spec.ts`, E2E

Bảng 5: Truy vết các yêu cầu MVP

Những chỗ tài liệu trong kho chưa khớp thực tế (cần sửa ở bản sau). (1) `traceability-matrix.md` còn ghi 74 test API trong khi thực tế là 112, và có một ô tự mâu thuẫn ("0/10 có kiểm thử tự động" nhưng chú thích trong ngoặc nói cả 10 đều có test). (2) Ma trận vẫn ghi FR-20 là kế hoạch, trong khi KDS đã merge vào `main`. Báo cáo này dùng số liệu đo lại, không dùng số trong ma trận.

8 Lịch sử Git và thứ tự tạo phẩm

Đề yêu cầu tài liệu Task 1–3 có trong lịch sử git trước mã. Số liệu dưới đây lấy từ `git log` trên `origin/main`.

Ngày (2026)	Sự kiện (commit đầu tiên của tạo phẩm)
26/10	Khung thư mục, CI; **đưa backend và frontend từ Assignment 1 vào** `apps/api`, `apps/web`
27/10	Đồng bộ thêm tính năng từ Assignment 1, workflow CD, backlog MoSCoW
30/10	Task 1: bối cảnh và phạm vi (`context-and-scope.md`), SRS giao diện, user story
01/11 – 07/11	Task 2: Activity-01, Class-01 (01/11), Sequence-01 (07/11); Task 1.3: use-case checkout (06/11)
03/11 – 04/11	Task 3: ADR 001, 004, kiến trúc frontend (03/11), Deployment diagram (04/11), SRS backend (04/11)
Từ 11/11	Sprint 2 hoàn thiện, E2E (12/11), Lighthouse (13/11), nền Sprint 3 và KDS

Điều cần nói thẳng. Mã nền của MVP (gồm cả backend đặt đơn, thanh toán và frontend) được đưa vào kho ngày 26/10 từ Assignment 1, *trước* các tài liệu yêu cầu (từ 30/10). Nên lịch sử git của kho này **không** thể hiện đúng thứ tự "tài liệu → mã" mà đề đòi hỏi cho phần mã nền. Các tạo phẩm tài liệu (Task 1–3) vẫn được commit đầy đủ, có Pull Request riêng, và đã được đối chiếu, sửa cho khớp mã ở các PR #117–#144. Báo cáo nêu rõ điểm này thay vì mô tả là đúng quy trình.

8.1 Đóng góp

Số liệu tính trên `origin/main` ngày 14/11/2026 (trước các commit Sprint 3–4 chưa đẩy lên). Tài dùng hai tên hiển thị git (`Tài Trần`, `TeoThanhLich`) với cùng một địa chỉ email nên được gộp theo email.

Nội dung	Cách tính	Long	Tài
Pull Request đã merge	tiền tố nhánh `long/`, `tai/` (74 PR)	21	53
Commit không tính merge	theo email tác giả	84	31
Dòng thêm / xóa	trong `apps/`, `docs/`, `tests/`	+35.891 / −11.320	+5.255 / −589

Số dòng của Long lớn vì gồm đợt đưa mã Assignment 1 vào ngày 26/10. Phần lớn Pull Request tài liệu (Task 1–3, ADR, truy vết, release notes) do Tài thực hiện; backend, CI/CD, E2E và triển khai do Long thực hiện.

9 Hạn chế và hướng phát triển

- Thứ tự "tài liệu trước mã" không thể hiện được cho phần mã nền (mục 8).

- FR-11 đến FR-19 và FR-21 (voucher, hủy đơn, đánh giá, báo cáo, kho, quản lý menu và nhân sự, mã nhận hàng) chưa merge vào `main` tại thời điểm viết; chỉ FR-20 (KDS) đã merge. Các phần này được làm ở nhánh riêng và phải được đẩy lên, thêm test, rồi mới được tính là hoàn thành.
- NFR-05 chưa đo hết (chưa đo đổi size/topping trong 50 ms và luồng menu đến thanh toán trong 60 giây); NFR-04 còn lỗi tương phản và tên liên kết.
- Backend production chạy trên gói miễn phí nên khởi động nguội mất vài giây đến hơn một phút ở lần gọi đầu.
- Các số liệu kiểm thử trong `traceability-matrix.md` và `release-notes.md` cần cập nhật theo số đo mới.

Tài liệu

[1] Đề bài lớn *BrewLite – Assignment 2: Requirements to Deployment*, Trường Đại học Sài Gòn, 2026.

[2] NestJS Documentation, https://docs.nestjs.com.

[3] Prisma ORM Documentation, https://www.prisma.io/docs.

[4] Next.js Documentation, https://nextjs.org/docs.

[5] PlantUML, https://plantuml.com.

[6] M. Nygard, *Documenting Architecture Decisions*, 2011.

[7] Google, *Lighthouse*, https://developer.chrome.com/docs/lighthouse.